

Criterion B: Design

GUI Diagrams

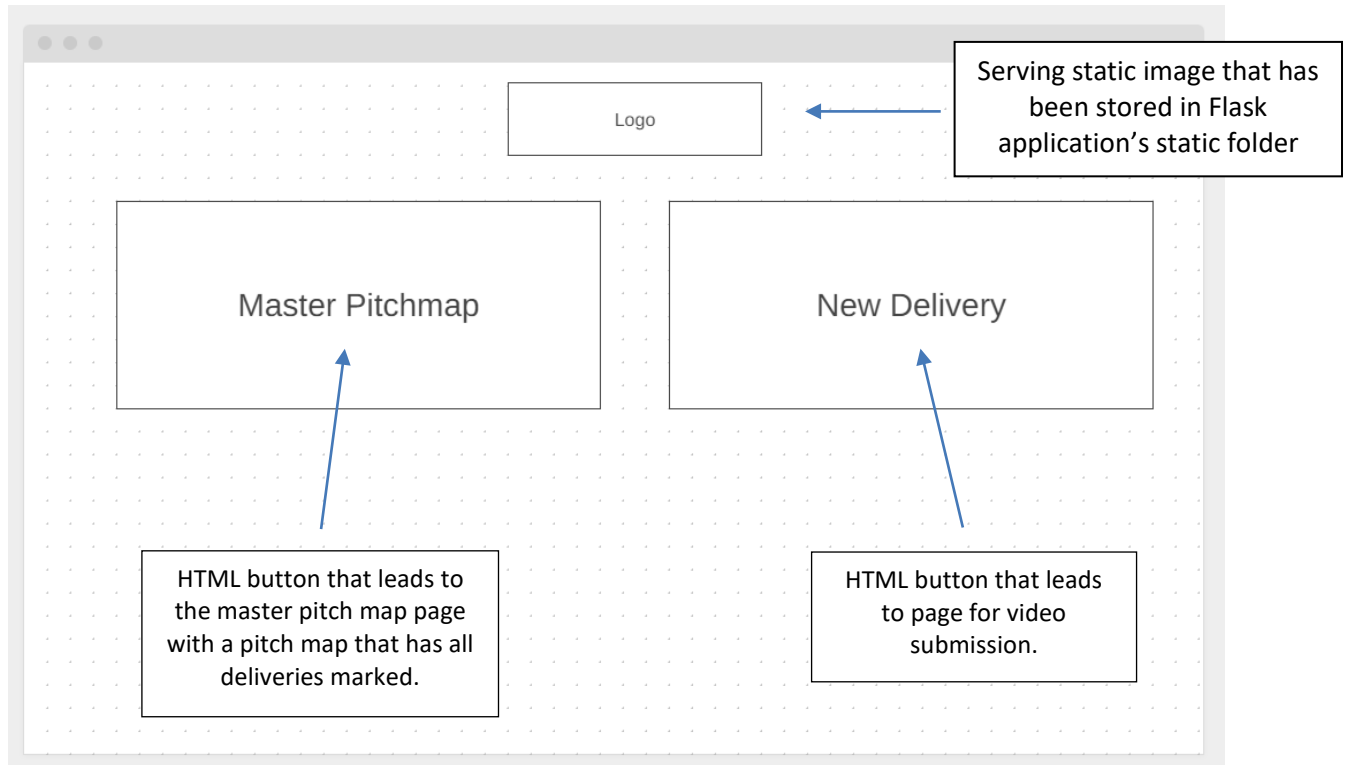


Figure 1: Initial Landing Page for the coach (homepage.html)

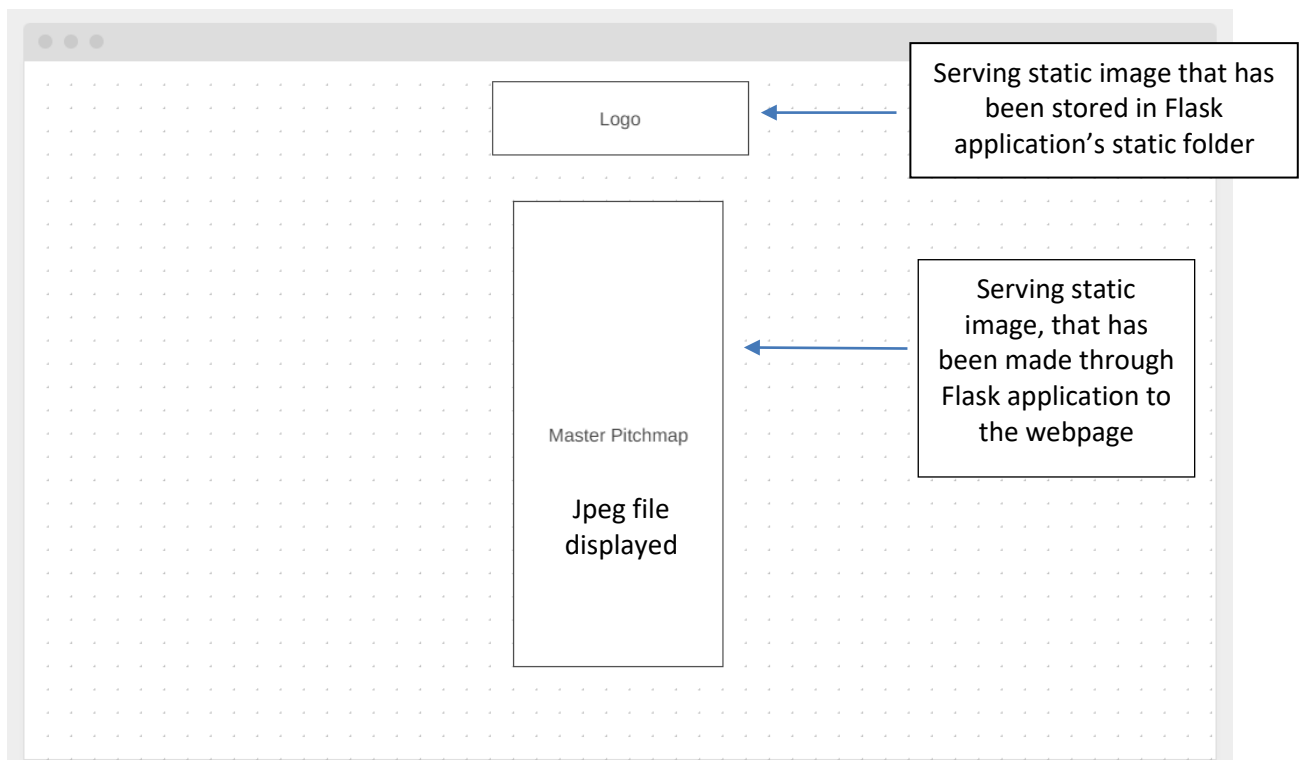


Figure 2: Master Pitch map page (Oldersessions.html)

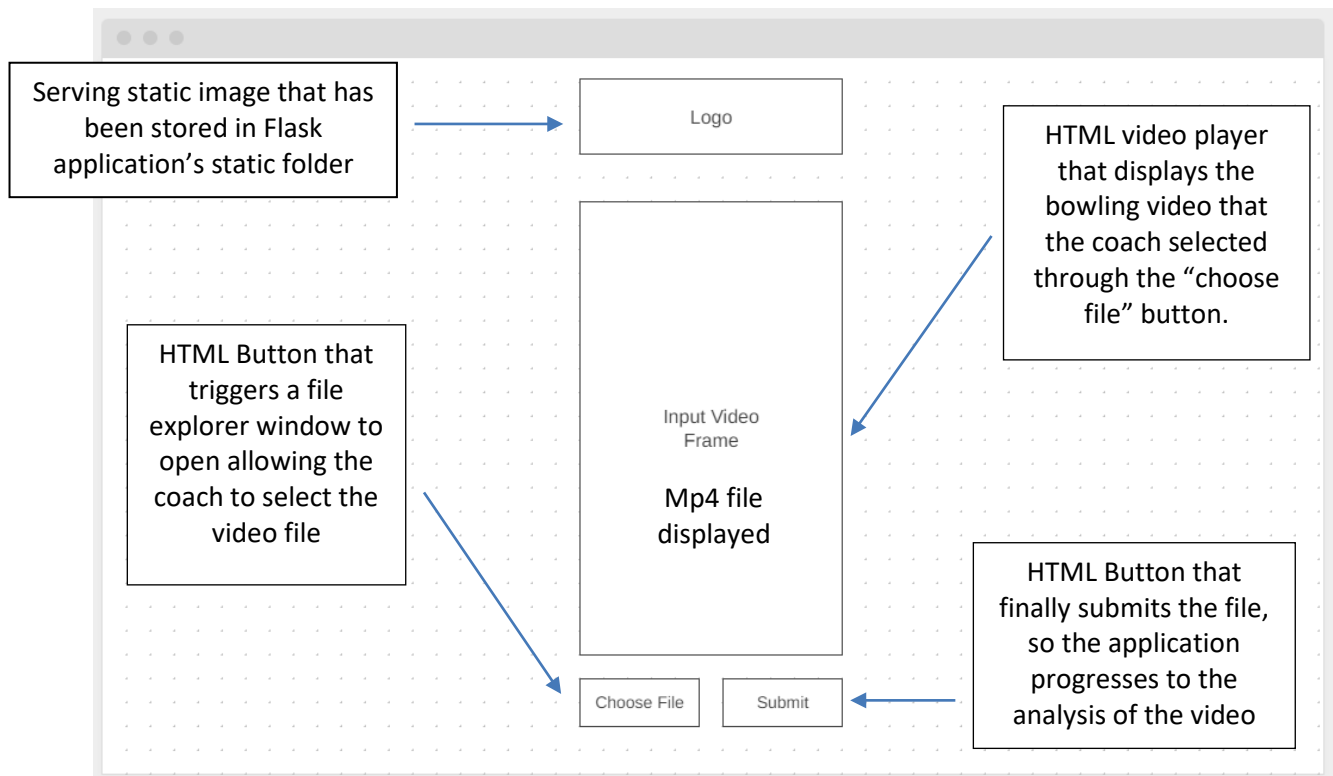


Figure 3: The Video Submission Page (Videosub.html)

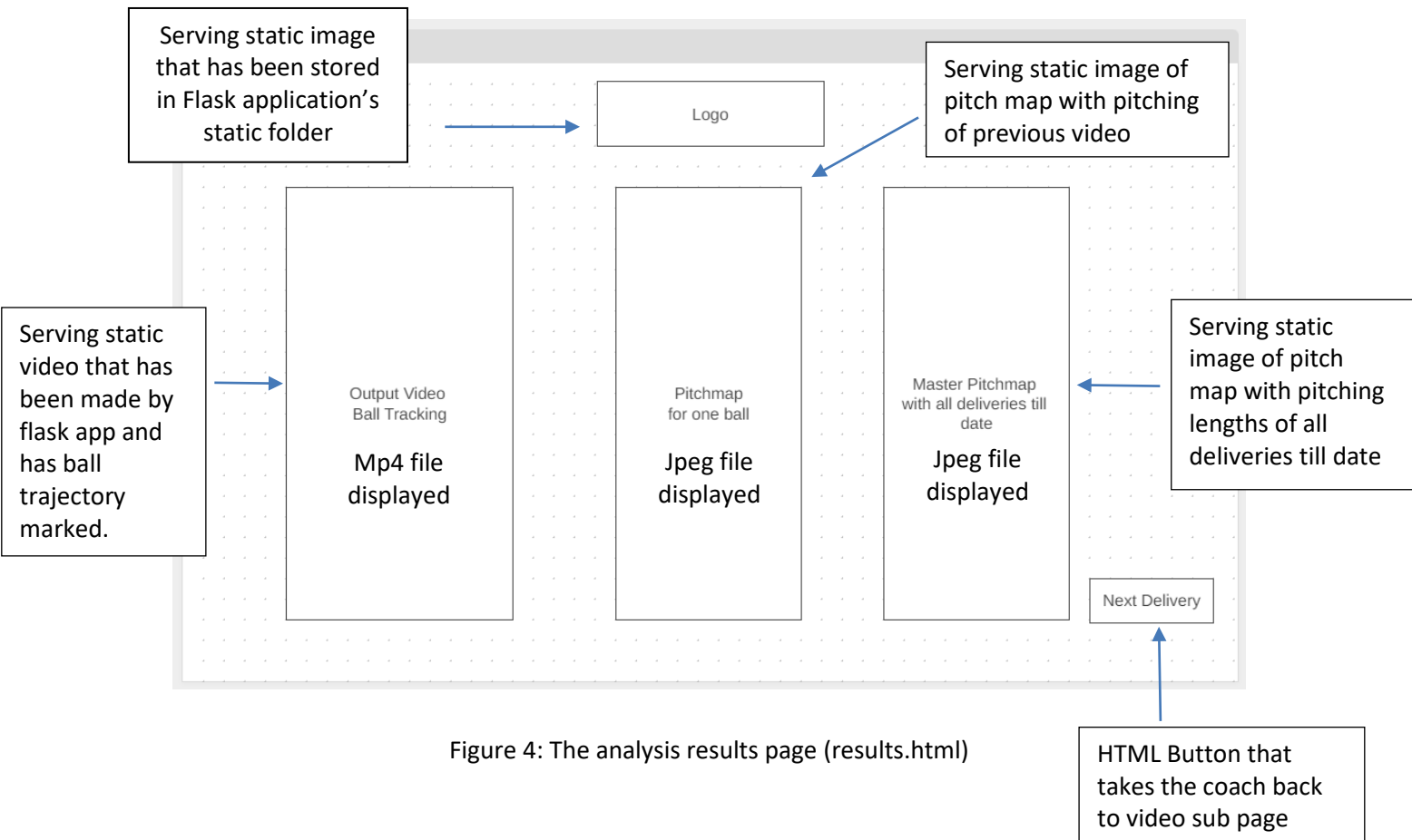
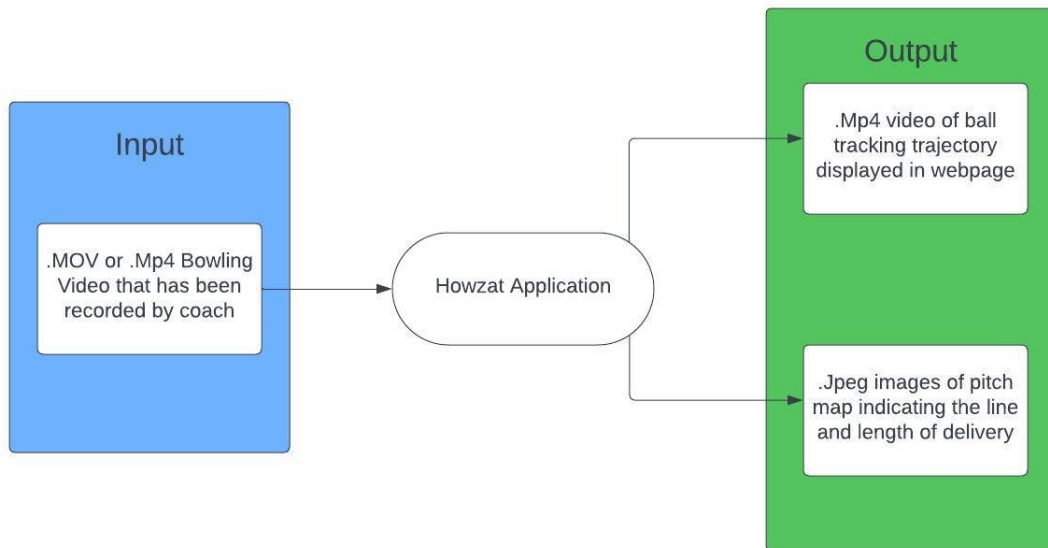
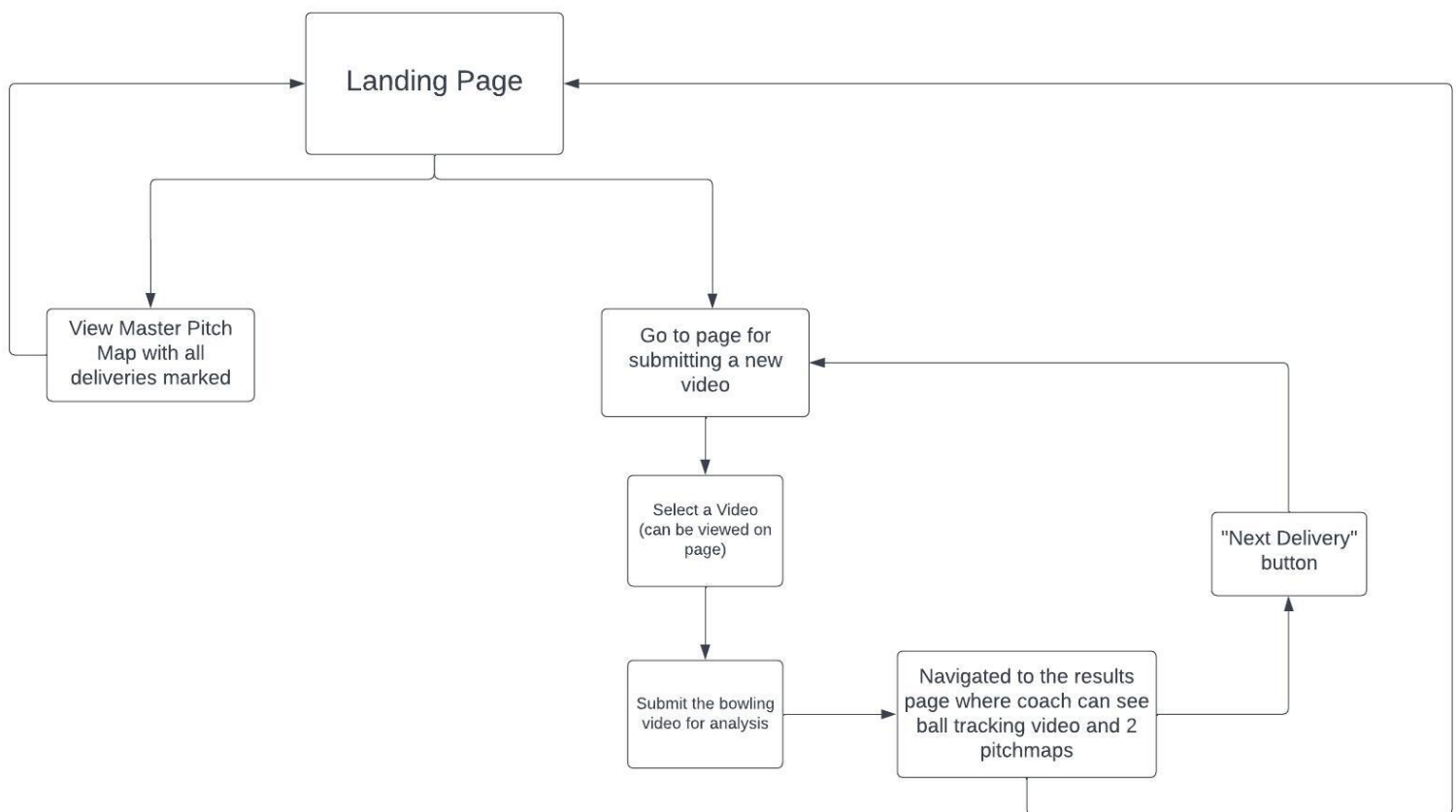


Figure 4: The analysis results page (results.html)

Context Diagram

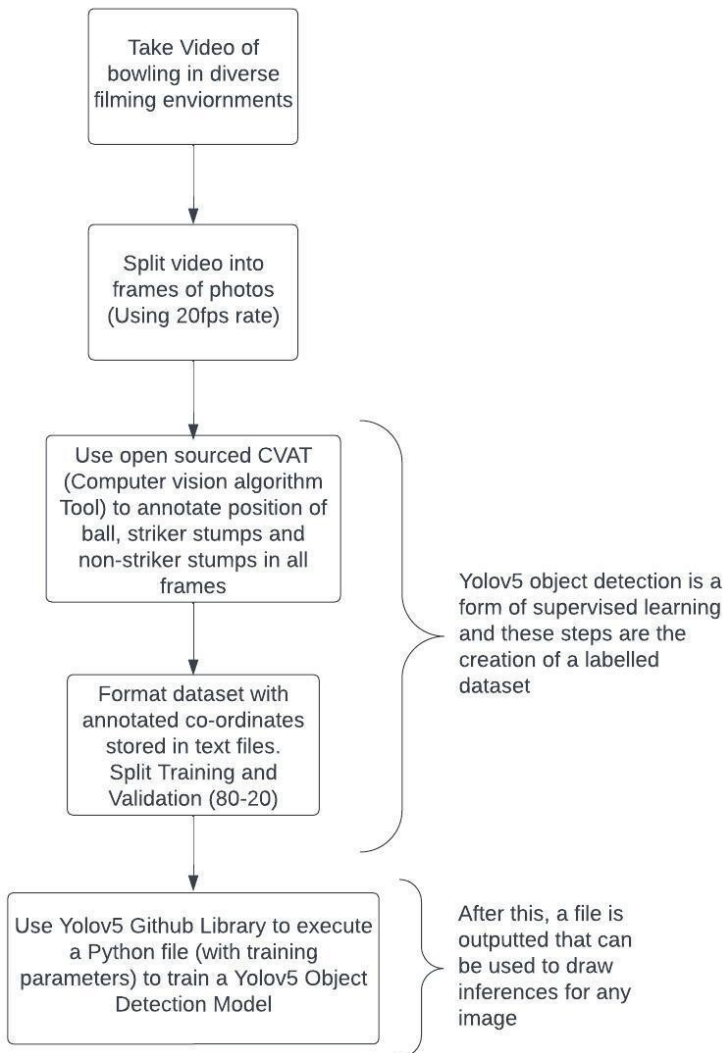


Navigation Diagram

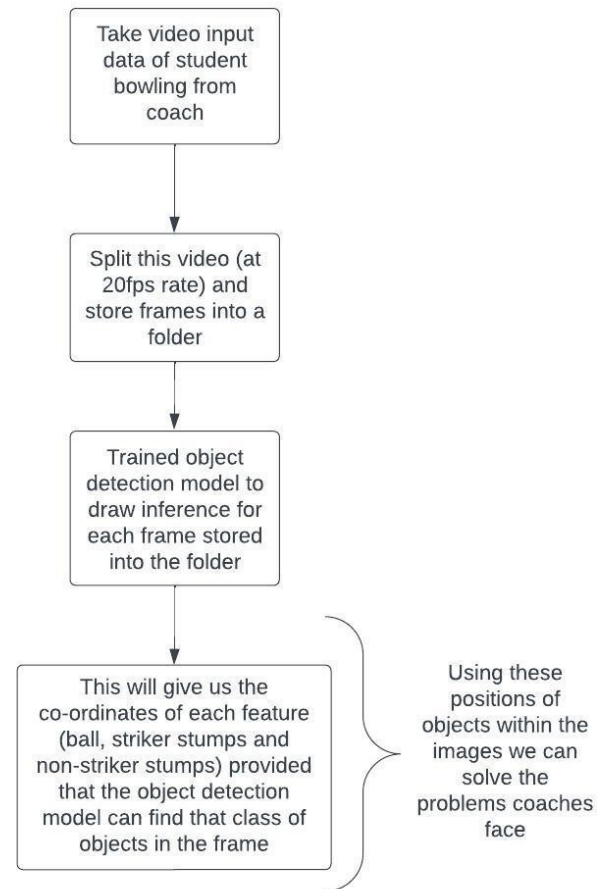


Computer Vision Process flows

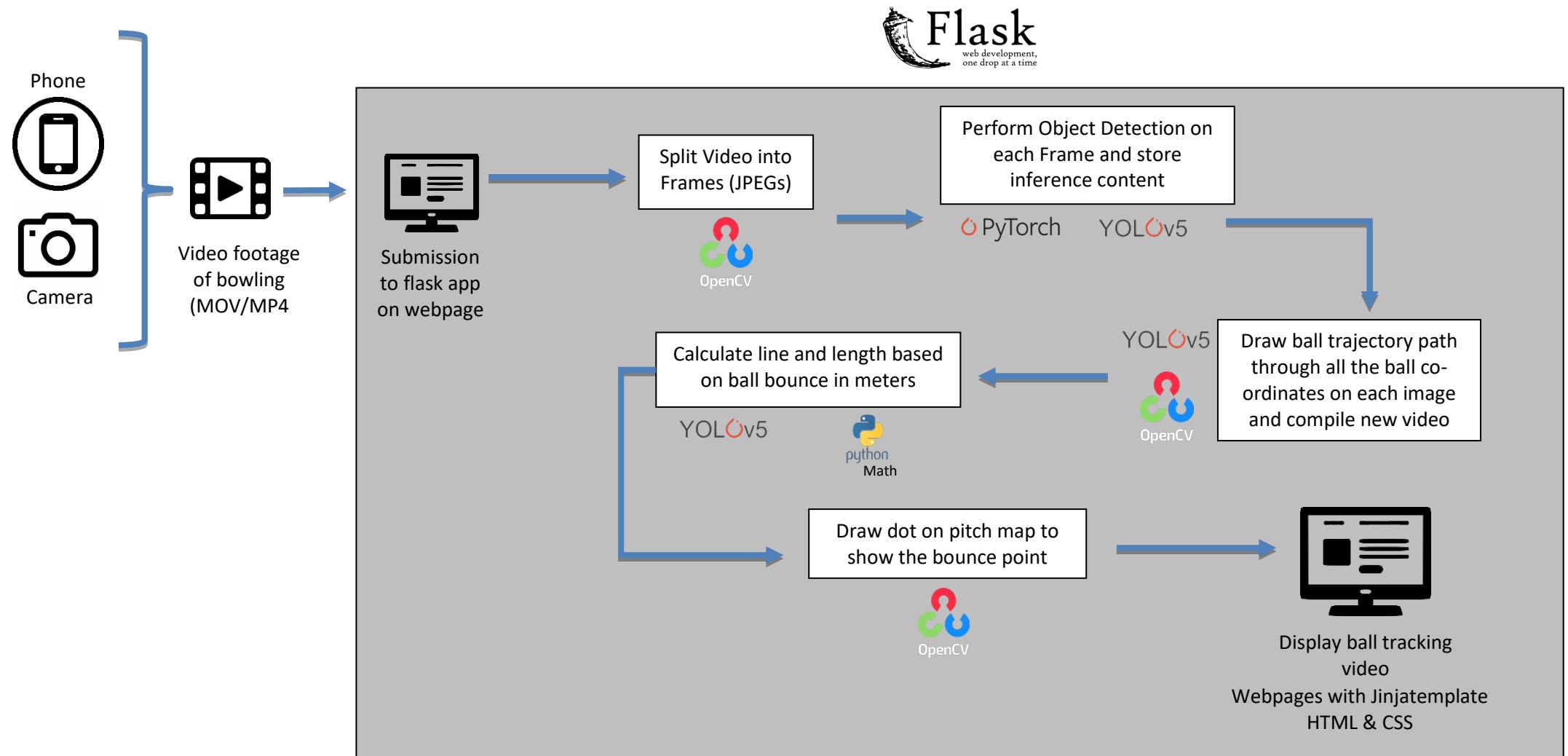
Object Detection Model Training



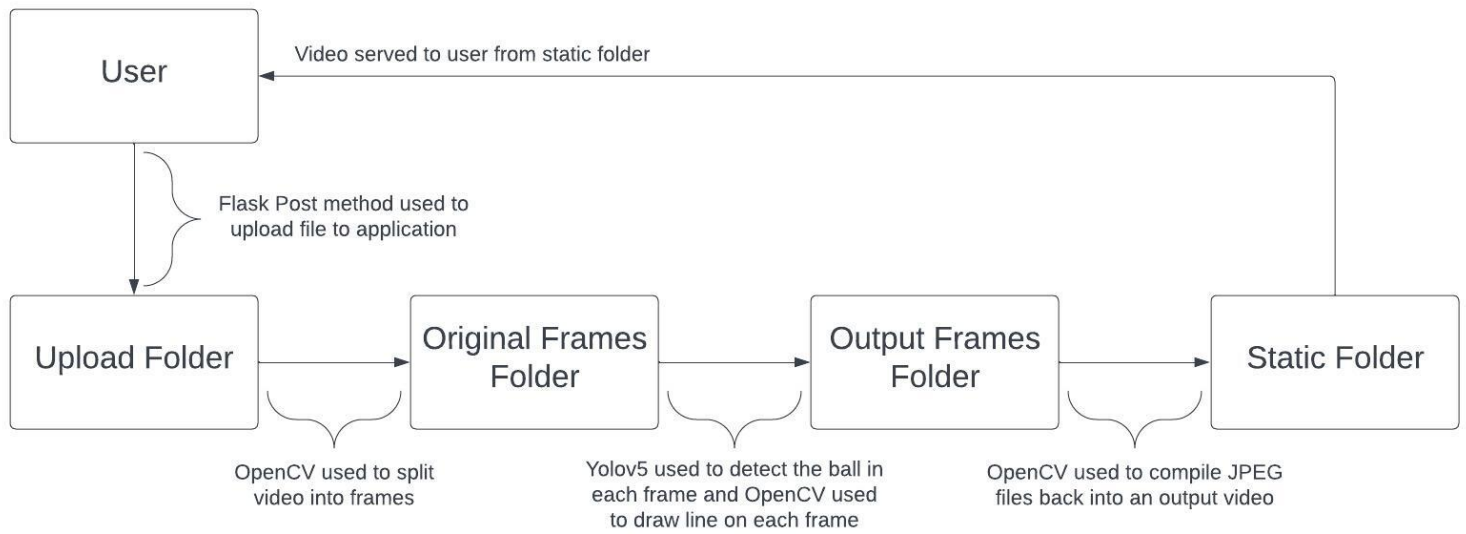
General Inference Process



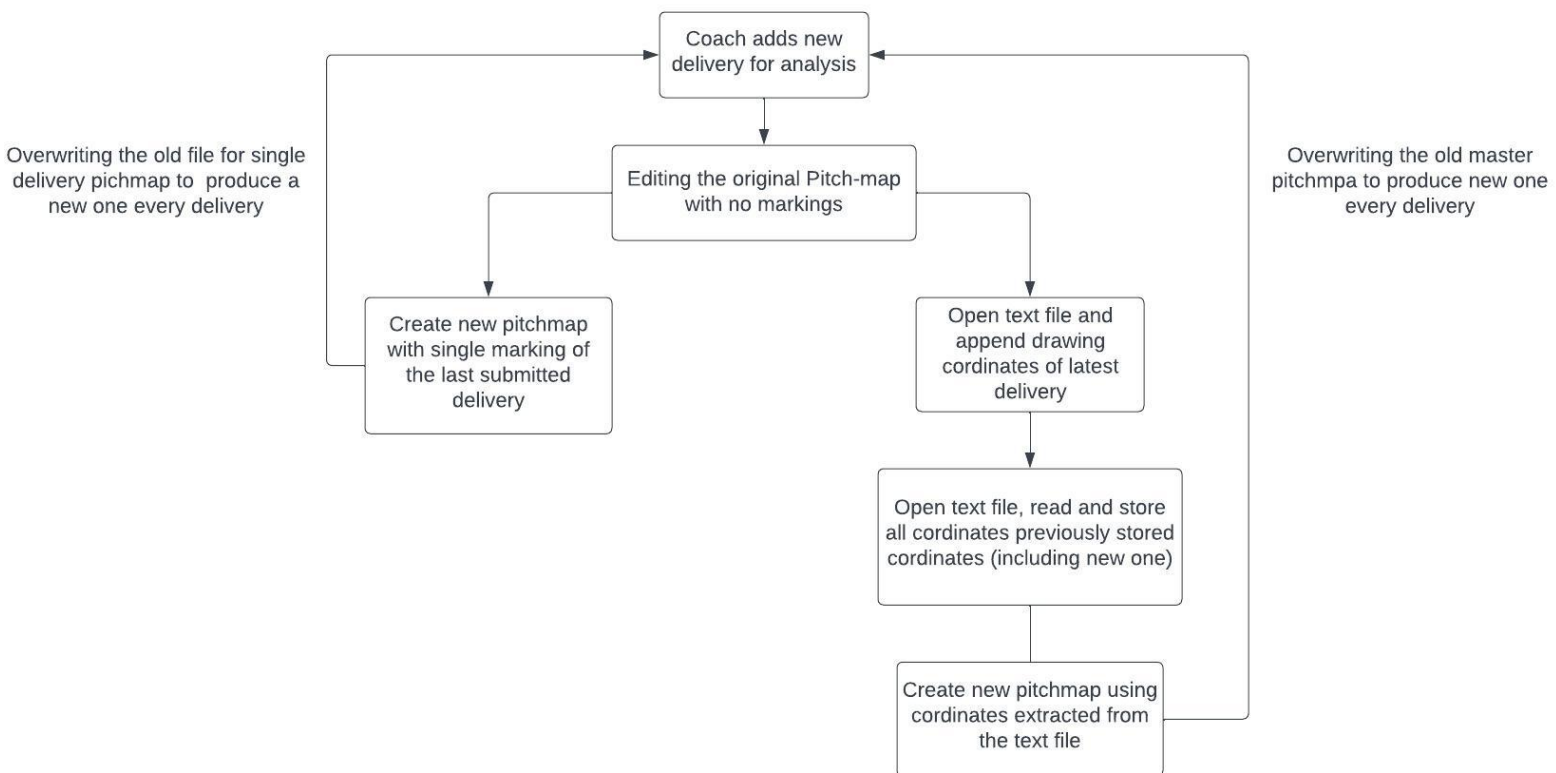
Application Architecture Diagram



Ball Tracking video File Handling Diagram



Pitch-map & Master Pitch-map File Handling



Length Analysis Calculation Pseudocode

```
// initialising new arrays for storing object detection results

S_STUMPS_HEIGHT = newarray() //array will store height of close stumps
NS_STUMPS_HEIGHT = newarray() //array will store height of far stumps
S_BASE = newarray() //array will store y-coordinate of base of close stumps
NS_BASE = newarray() //array will store y-coordinate of base of far stumps

#object detection inference and storage

For each image in the split folder:
    detection = obj_detect_function()
    For I = 0 to (objects detected in the image): //looping I to figure out index of specific class detected
        if detection[i] == 'Stumps':
            ymin_stumps = detection[i]['ymin']
            //Yolov5 outputs coordinates of bounding box in following format, ymin, ymax, xmin, xmax
            ymax_stumps = detection[i]['ymax']
            S_stump_height = (ymax_stumps - ymin_stumps)
            Append S_stump_height to the S_STUMPS_HEIGHT array
            Append ymax_stumps to S_BASE array

        if detection[i] == 'NS_stumps':
            ymin_NS_stumps = detection[i]['ymin']
            ymax_NS_stumps = detection[i]['ymax']
            NS_stump_height = (ymax_stumps - ymin_stumps)
            Append NS_stump_height to the NS_STUMPS_HEIGHT array
            Append ymax_NS_stumps to NS_BASE array

//after appending the coordinates from each image we need to take average
//this average will give us more accurate coordinates if bounding boxes are different
AVG_stump_height = (sum(S_STUMPS_HEIGHT))/(len(S_STUMPS_HEIGHT))
AVG_NS_stump_height = (sum(NS_STUMPS_HEIGHT))/(len(NS_STUMPS_HEIGHT))
AVG_NS_Base = (sum(NS_BASE))/(len(NS_BASE))
AVG_S_Base = (sum(S_Base))/(len(S_Base))
distance = (AVS_NS_Base - AVG_S_base) //pixel distance between the two stumps

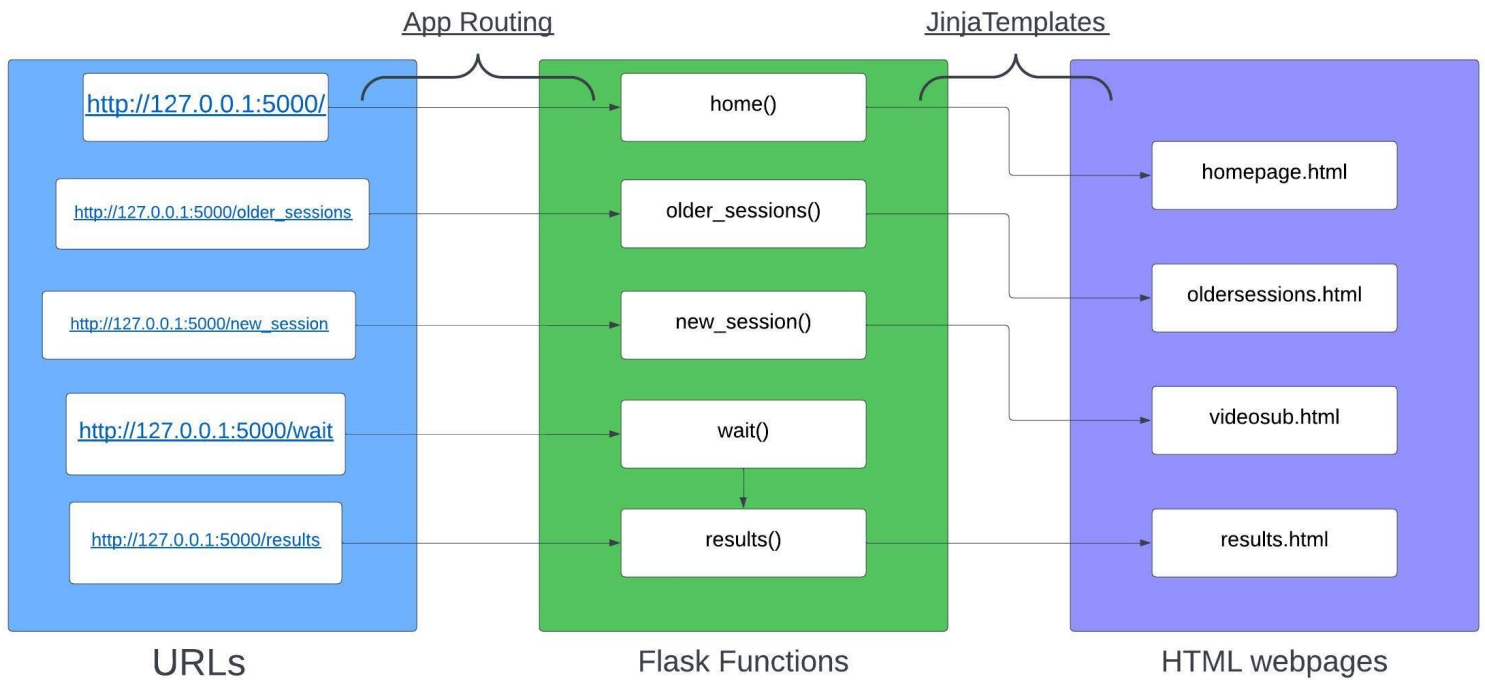
//create stump pixel height vs pitch distance function
//assume the following coordinates (0, AVG_NS_Base) because here the distance between the stumps is 0
//then assume another coordinate as (distance, AVG_S_Base)
//with these we can develop the function by calculating the gradient
Gradient = (AVG_NS_base - AVG_S_Base)/(0-distance) // using the formula m = y2-y1/x2-x1

//function: stumps pixel height = Gradient * (stump pixel distance) + AVG_NS_Base using y = mx+c

//To calculate the height of the far stumps if reconstructed at where the ball bounces
//we will substitute the pixel distance between ball bounce and the close stumps into the function

reconstructed_stump_height = Gradient((NS_Base - BounceY) - stump_to_stump_distance) + AVG_NS_Base
//meter distance can now be calculated using proportion of both stump heights
meter_dist = (reconstructed_stump_height/AVG_NS_stump_height)/(20.16)
//20.16 is the real distance in meters of the pitch between the stumps
```

Flask Application Structure



Flask Functions and Operation Plan

Possible Flask Function	Operation
Home()	No server-side scripting Returns ' <i>homepage.html</i> ' that acts as a landing page and redirects the coach to other URLs
Older_sessions()	No server-side scripting Returns ' <i>oldersessions.html</i> ' that serves a static image of the master pitch map that shows the length of all deliveries till date
New_session()	Utilises 'GET' and 'POST' HTTP requests to allow user to submit a video for analysis Uses OpenCV to split video and draw on frames and compile frames Uses Yolov5, Pytorch and Pandas to do object detection inferences and store co-ordinates of object detection Uses OS library for file handling Returns ' <i>videosub.html</i> ' page that allows user to select a video and watch it on the webpage

Wait()	Only function that is not attached to a HTML webpage. This function acts a blocking call: the user is kept on the video submission page whilst the wait function ensures that analysis has been done on the new delivery video Once it has made sure that the video has been analysed the user is directed to the results
Results()	No server-side scripting Returns '<i>results.html</i>' that serves 2 static images of the current ball and master pitch map and 1 video of ball tracking

Test Plan

Action/Criteria to be tested	Method of Testing
Coaches should be able to select a video file of their student's bowling that they wish to submit to the application for analysis	Use "Choose File" button on the video submission page
	<i>Check</i> that a file explorer window is opened
	<i>Check</i> that the user can select any video file extension that is compatible with OpenCV2
Coaches should be able to view their selected video on the webpage with all abilities of video playing, such as pause/play/forward/rewind. However, the coach should also have the ability to slow the speed of the video down. (split up)	Use the "Choose file" button on the video submission page to select the video file
	<i>Check</i> that the video appears in the video frame on the page
	<i>Check</i> that video can be paused and played
	<i>Check</i> that video can be forwarded and rewind
	<i>Check</i> that video can be slowed down into: 0.75x, 0.5x, 0.25x
Coaches should be able to upload a video, with a maximum length of up to 5 seconds to the application. (talk about the error message)	Use the "Submit File" button on the video page to submit the selected
	<i>Check</i> video upload by <i>checking</i> if file has been saved in "Upload Folder" in the Flask application directory
	<i>Check</i> that an error message appears if video is longer than 5 seconds
	<i>Check</i> that the error message prompts the user to resubmit a shorter video
The application should be able to produce an output video that clearly shows the trajectory path of the ball using a visible white line.	After video submission wait for the analysis and redirection to the results page
	<i>Check</i> that an output ball tracking video is displayed in the video frame
	Play the video to visually <i>check</i> that white line is drawn through the path of the ball.

In addition to a line that displays the path/trajectory of the delivery, the output video should contain a red dot that shows where the ball bounces.	Once the result page has loaded play the video
	Visually <i>check</i> that red dot is approximately where the ball makes contact with the ground
The application should be able to determine how many meters the ball bounces away from the stumps of the striker.	Once the result page has loaded
	<i>Check</i> under the “current delivery pitchmap” image for a numeric value that represents how many meters the ball bounced away from the striker stumps
	Verifying the validity of this length calculation is out of the scope of this project
The application should be able to produce a pitch map graphic on which the bounce spot is shown by a white dot drawn at the bounce position of the specific delivery in the submitted video.	Once the result page has loaded
	<i>Check</i> that a Jpeg pitch map image is displayed
	<i>Check</i> the pitchmap has one white dot that shows the length of where the ball bounced.
	Testing the validity of the bounce spot is not a part of the scope of this project.
The application should be able to produce a pitch map that displays the bounce spot of all deliveries that have been uploaded to the application to date.	Once the results page has loaded.
	<i>Check</i> that a master pitch map is shown
	<i>Submit</i> an X number of videos and verify that there are an X amount of dots on the master pitch map